fetchmany(2) → how many rows

VVI for a single param

use (age,) , is mandatory as it is a python tuple

Notebook

notebook is a special file (.ipynb) that lets you add notes, eqn, images, charts etc

Jupyter notebooks → using anaconda

Numpy

import numpy as np

① creating array from list.

param is the list.

```
arr_1d = np.array([1,2,3,4,5])
```

List VS Array

Slower

```
py_list = [1,2,3]
print(py_list*2)   // o/p [1,2,3,1,2,3]
```

but array faster

```
arr = np.array(list)
print(arr*2)   // o/p [2,4,6]
```

Array from scratch

zeros = np.zeros((3,4)) → will create a 2D array of 3 rows 4 cols with 0.

.ones(( )) → same.

full = np.full((2,2), 7) → with any digits to create.

random = np.random.random((2,3))

// random array

.rand(10) → any 10 random array.

.arange()

arr = np.arange(1, 11) → step.

O/P [1,2,3,4,5,6,7,8,9,10]

Vector – 1D array.

Matrix – 2D array.

tensor → multi-dimension array.

(.linspace(0,1,5) O/P will be 5 no between 0 & 1 with equal interval)

Array properties

arr = np.array([[1,2,3], [4,5,6]])

arr.shape // → (2,3)

.ndim // 2 [dimension of the array]

.size // total elements present

.dtype // datatype ← int64, float (U>)

.sum() // total sum

Array reshape

arr = np.arange(12)

rearr = arr.reshape((3,4))

O/P arr will be converted to rearr dimension

flattend = rearr.flatten( )

→ 2D, 3D to 1D.

Transpose

transpose = rearr.T

Interview

Ravel vs flattened

Array operations.

arr = np.arange(11)

1) Slicing

arr[2:7:] ← step
↓ start ↓ stop

2) Sorting

np.sort(unsarr)

0 → column wise
r = row wise

for 2D array, np.sort(arr_2D_un, axis = 0)

filter.

even_arr = arr[arr % 2 == 0]
↳ condn

o/p 2 4 6 8 10

fancy indexing & np.where()

idx = [0, 2, 4]
print(arr[idx]) // will return the values @ the indices.

where_res = np.where(arr > 5)
print(where_res) // [5,6,7,8,9] → >5 indexes are returned.

print(arr[where_res]) o/p → values @ those indexes.

con_arr = np.where(arr > 5, "true", "false")

add & remove

a1 = np.array([1,2,3])
a2 = ---([4,5,6])
a = np.concatenate((a1, a2))
o/p [1,2,3,4,5,6]

ar = np.delete(a1, 2)
o/p [1,2]

vstack vs hstack

```
ori = np.array([[1,2],[3,4]])
new_row = np.array([[5,6]])
```

// to insert a new row, use vstack.

```
with_new_row = np.vstack((ori, new_row))
print(r)
```

o/p

```
[[1, 2]
 [3, 4]
 [5, 6]]
```

// to insert a new col, use hstack.

```
new_col = np.array([[7],[8]])
with_new_col = np.hstack((ori, new_col))
```

o/p

```
[[1 2 7]
 [3, 4 8]]
```

Exercise

Advance operation with business example.

```
sales_data = np.array([
    [1, 150000, 180000, 220000, 250000],
    [2,
     :
     :
    [5 ]
])
print("zomato sales analysis ==")
```

1. arr.astype(datatype)
   ↳ converts an existing array to a specific dtype.

2) np.eye(4,4) ↳ used to create a identity matrix.

3) Count no equal to 2.

```
print(np.count_nonzero(arr==2))
```

↳ used to count to no of elements satisfying a condn.

④ Find all unique values in array.

```
print(np.unique(arr))
```

↳ returns distinct, By default it returns them in sorted order.

⑤ Counting occurances.

```
arr = np.array([1,2,2,3,4,4,4,5])
unique_values, counts = np.unique(arr, returns_count=True)
print("Unique values:", unique_values)
print("Frequencies:", counts)
```

⑥ Find ~~max~~ Index

```
np.argmax(arr)
```

⑦ ~~add~~. np.add(arr, 10)
.multiply(arr, 10)
.dot(a,b) → matrix multiplication
print(a @ b) →
.matmul(a,b) →

⑧ .diagonal().

⑨ Find duplicates.

```
uv, count = np.unique(arr, returns.count=True)
dup = uv[count>1]
print(dup).
```

(10) Print 3 max values.

```
top_3 = np.sort(arr)[-3:][::-1]
print(top_3)
```

(11) Diff betn consecutive values.

```
changes = np.diff(arr)
```

Real-world practise.

```
arr = np.array([
        [78, 85, 90, 66],
        [ - - - - ],
        [ - - - - ],
        [ = = = = ]])
```

a) print each st^ avg marks.

```
ans = np.mean(arr, axis = 1)
```